FIT2099 Assignment 3 — REQ5 Design Rationale

REQ5 implements a real-world weather system that fetches live data from the OpenWeatherMap API and applies environmental effects to the game map. The system is divided into five key design decisions, each evaluated against a simpler alternative.

A. Representing Weather Effects (Threshold-Triggered)

Design 1: Implement all threshold-triggered weather effects (Heatwave, Fungal Bloom, Storm) as conditional blocks inside a single applyEffects() method in WeatherSystem, using numeric if-checks for each effect type.

Pros	Cons
Trivial to implement; all effect logic is visible in one place.	Adding a new weather effect requires modifying `WeatherSystem` directly, violating the Open-Closed Principle — the class must be changed every time the game grows.
	The method grows unbounded as more effects are added, becoming hard to read and test in isolation (violation of Single Responsibility Principle).
	Each effect is tightly coupled to `WeatherSystem`; changing one effect's threshold requires touching a class that orchestrates the entire pipeline.

Design 2 (Chosen): Extract each effect into a concrete class implementing the WeatherEffect interface, which contracts two methods: shouldActivate(WeatherReport) and apply(GameMap, Location). WeatherSystem iterates over a List<WeatherEffect> and calls both methods polymorphically.

Pros	Cons
Open-Closed Principle: a new effect (e.g. `BlizzardEffect`) is added by creating a new class and registering it — no existing code changes.	Introduces more classes; a small project may find this over-engineered relative to the number of effects actually present.
Single Responsibility: `HeatwaveEffect` is the only class that knows the 32°C threshold and how to spawn fire tiles — it cannot be affected by changes to humidity logic in `FungalBloomEffect`.	

Each effect is independently unit-testable with a mock `WeatherReport` — no real HTTP call is needed.	
Dependency Inversion: `WeatherSystem` depends on the `WeatherEffect` abstraction, never on `HeatwaveEffect`, `StormEffect`, etc.	

Chosen: Design 2.

The three concrete effects (`HeatwaveEffect`, `FungalBloomEffect`, `StormEffect`) each own their own threshold constant and map-mutation logic. `WeatherSystem` is closed for modification but open for extension — adding a fourth effect requires zero changes to the orchestration layer. Each effect class has a single, well-named responsibility, reducing connascence of meaning: a reader of `HeatwaveEffect` never needs to know that Storm also exists.

B. Representing Climate Zones (Passive Always-On Effects)

Design 1: Encode zone detection and passive effects as a switch-on-integer block inside WeatherSystem, branching on the player's x-coordinate to select a city URL and apply a hard-coded terrain change.

Pros	Cons
Simple; no additional classes required for a fixed set of three zones.	Adding a fourth zone requires editing `WeatherSystem`, violating the Open-Closed Principle.
	City coordinates, column ranges, and terrain mutation logic are all mixed into one class — a clear violation of Single Responsibility and high connascence of position (the order of switch cases matters).
	Dynamic URL generation is impossible to unit-test in isolation when it is embedded in the same method that fires HTTP requests.

Design 2 (Chosen): Create an abstract class WeatherZone with four abstract methods: buildApiQuery(String), containsPlayerX(int), applyPassiveEffect(GameMap, Location), and getZoneName(). Three concrete subclasses (AridZone, TemperateZone, HumidZone) each own their city coordinates, column range, and unique passive effect.

Pros	Cons

Open-Closed: a new zone (e.g. a Polar Zone for Reykjavik) is a new subclass only — `WeatherSystem` receives it via its constructor list and needs no modification.	Abstract class inheritance introduces tighter coupling than an interface — subclasses cannot extend any other class. Accepted because zones share enough structural identity (API URL, column range, passive effect) to justify a common abstract base.
Each zone knows its own real-world city coordinates, keeping geographic knowledge collocated with the class that uses it (low connascence of identity).	
Dependency Inversion: `WeatherSystem` holds a `List<WeatherZone>` — never a reference to any concrete zone class.	
Abstract class over interface chosen so that future zones can share utility code (e.g. a common URL template method) without repeating it, satisfying DRY.	

Chosen: Design 2.

`AridZone` evaporates Puddles and dilutes ToxicWaste, `TemperateZone` accelerates flora growth, and `HumidZone` suppresses fire and amplifies fungal spread — three completely different behaviours, each isolated in its own class. The `containsPlayerX` contract makes zone activation a pure boolean query, enabling trivial unit tests with no map setup required.

C. Parsing the API JSON Response Without instanceof

Design 1: Parse all four fields (temperature, humidity, wind speed, condition) inside a single buildReport(String) method in WeatherSystem. Store extracted values as Object in a map, then use instanceof to cast each value to the correct type before constructing WeatherReport.

Pros	Cons
All parsing logic is in one place — easy to find for a first-time reader.	`instanceof` is a code smell: it couples the caller to the runtime type hierarchy, makes adding a new field error-prone, and was explicitly forbidden by the project's design rules.
	Adding a fifth field (e.g. UV index) requires editing `WeatherSystem` and extending the instanceof chain — violates Open-Closed Principle.

	The method carries high connascence of type: the caller must know which `Object` is a `Double` and which is an `Integer`.

Design 2 (Chosen): Introduce a generic interface WeatherDataExtractor<T> with methods extract(String) (returns the typed value) and populateReport(WeatherReportBuilder, String) (writes directly to the builder). A mutable WeatherReportBuilder accumulates parsed values field by field. WeatherSystem.buildReport() simply iterates the extractor list and calls populateReport on each.

Pros	Cons
Eliminates all `instanceof`: each extractor knows its own concrete type and calls the correct typed setter on the builder (`setTemperature(double)`, `setHumidity(int)`, etc.).	Introduces more classes (four extractors + one builder) for a problem that could be solved with fewer lines if `instanceof` were permitted.
Open-Closed: adding a UV index field means creating `UvIndexExtractor` and adding `setUvIndex` to the builder — `WeatherSystem` is unchanged.	
Builder pattern (KISS): `WeatherReportBuilder` has safe defaults for all fields, so a partial API response (missing humidity, for example) produces a valid `WeatherReport` with no NPE risk.	
Each extractor is independently unit-testable with an inline JSON string — no network, no mocking.	

Chosen: Design 2.

The generic type parameter `T` on `WeatherDataExtractor` enforces type safety at compile time; the caller never sees a raw `Object`. The Builder pattern keeps `WeatherReport` immutable after construction while allowing partial population — a classic KISS solution to a multi-step initialisation problem. Connascence of name is reduced: only `TemperatureExtractor` knows the JSON key `"main.temp"`; no other class contains that string.

D. Orchestrating the Weather Pipeline (Dependency Inversion)

Design 1: WeatherSystem directly instantiates its zones and effects (new AridZone(), new HeatwaveEffect(), etc.) in its own constructor. It also hard-codes the API key as a string constant.

Pros	Cons
No configuration needed at the call site — the system is self-contained.	Violates Dependency Inversion Principle: `WeatherSystem` (high-level module) depends on all concrete zone and effect classes (low-level modules).
	Unit testing is impossible without triggering live HTTP calls and real terrain mutations, because the concrete classes are baked in.
	A hard-coded API key is a critical security vulnerability and would be committed to version control.

Design 2 (Chosen): WeatherSystem receives a List<WeatherZone> and a List<WeatherEffect> through its constructor (constructor injection / Dependency Inversion Principle). The API key is never hard-coded: loadApiKey() reads first from the OPENWEATHER_API_KEY environment variable, then from a .env file, and falls back to null (safe default mode) if neither is present.

Pros	Cons
Dependency Inversion Principle: `WeatherSystem` depends only on `WeatherZone` and `WeatherEffect` — it has zero import of any concrete zone or effect class.	The caller (`EclipseNebula`) must assemble the list of zones and effects, introducing a composition root. This is a normal trade-off in DIP-based designs.
API key security: the key lives in a `.env` file excluded from git (`.gitignore`), preventing accidental exposure in version control.	
Testability: tests can inject zero effects or a single stub zone without touching the HTTP stack.	
Open-Closed: swapping in a different weather data source (e.g. a mock server) requires no change to `WeatherSystem` — only the injected objects change.	

Chosen: Design 2.

The composition root in `EclipseNebula` is the sole place where concrete classes are mentioned; everywhere else in the weather package, only abstractions appear. The API key loading follows a secure precedence chain (environment variable → file → default) that is standard practice for configurable secrets and directly addresses the OWASP guideline against

hard-coded credentials. The safe fallback ensures the game runs offline without crashing, improving robustness.

E. Integrating the Weather System with the Game Engine

Design 1: Trigger the weather update inside the tick() method of an existing Ground subclass, or inside Player.playTurn(), and call WeatherSystem.fetchAndApply() directly from there every N turns.

Pros	Cons
No new actors or classes needed — reuses the existing tick/playTurn machinery.	Violates Single Responsibility Principle: the `Player` or a `Ground` tile now also manages the weather pipeline, coupling two unrelated concerns.
	Ground tiles do not hold a reference to the entire map — fetching nearby locations or applying area-of-effect changes is architecturally awkward from a single tile's tick method.
	Tying the weather cycle to the player's turn makes it harder to later configure a different schedule (e.g. every 15 turns of any actor, not just the player).
	Misuse of the engine: Behaviours are the engine's intended mechanism for actor-driven periodic actions. Using `playTurn` for weather contradicts the engine's design intent.

Design 2 (Chosen): Introduce a dedicated WeatherController actor placed on a fixed tile in EclipseNebula. Its playTurn() delegates to a WeatherBehaviour, which implements the engine Behaviour interface. Every 15 turns WeatherBehaviour returns a WeatherAction (extends engine Action); on all other turns it returns a DoNothingAction. WeatherAction.execute() calls WeatherSystem.fetchAndApply().

Pros	Cons
Correct use of the engine: `Behaviour` is the engine's contract for deciding what action to take; `Action` is the contract for executing a game-turn effect. The weather system uses both correctly.	Adds two new classes (`WeatherController`, `WeatherBehaviour`) that may seem heavyweight for a periodic task. This cost is justified by design correctness and the testability gained.

Single Responsibility: `WeatherController` is the only actor whose sole purpose is weather scheduling; `Player` is untouched.	
The 15-turn interval is a single constant in `WeatherBehaviour` — tuning it does not touch any other class (low connascence of value).	
WeatherBehaviour finds the player by scanning for the `WORKER` ability, keeping the weather system fully decoupled from the concrete `Player` class (Dependency Inversion Principle).	
`WeatherAction` is independently unit-testable by calling `execute()` with a stub map.	

Chosen: Design 2.

Design 2 aligns with the engine's own Actor/Behaviour/Action architecture. `WeatherController` is an actor like any other — the engine's game loop advances it automatically, keeping weather updates synchronised with the game turn without any external timer or thread. The chain `WeatherController → WeatherBehaviour → WeatherAction → WeatherSystem` maintains the Dependency Inversion Principle at every level: each class depends on the abstraction immediately below it, never on a concrete implementation two levels down.

Summary of Design Principles Applied in REQ5

DRY: All JSON key strings and threshold constants appear once, in the class that owns them.

KISS: Builder with defaults eliminates null-guard repetition; `DoNothingAction` avoids the null-return anti-pattern.

Single Responsibility Principle: Each class has one reason to change — extractors parse, effects apply, zones define terrain, `WeatherSystem` orchestrates.

Open-Closed Principle: New effects, zones, and extractors are added without modifying existing classes.

Liskov Substitution Principle: All `WeatherZone` subclasses honour the abstract contract; `WeatherSystem` can substitute any zone without behavioural surprises.

Interface Segregation Principle: `WeatherEffect` and `WeatherZone` are kept separate; no implementing class is forced to implement methods irrelevant to its role.

Dependency Inversion Principle: `WeatherSystem`, `WeatherBehaviour`, and `WeatherAction` all depend on abstractions; concrete classes are assembled only at the composition root (`EclipseNebula`).